

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Угриновський Богдан

МЕТОДИЧНІ ВКАЗІВКИ

ДО ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ №1

З ДИСЦИПЛІНИ «КРОСПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

ДЛЯ СТУДЕНТІВ першого (бакалаврського) рівня вищої освіти

СПЕЦІАЛЬНОСТІ 121

«ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»

Львів - 2026

ЛАБОРАТОРНА РОБОТА №1

Тема. Вступне заняття. Інструктаж з техніки безпеки. Ознайомлення з середовищем розроблення. Створення та запуск першого проєкту, ознайомлення з базовою структурою проєкту.

Мета. Засвоїти основи роботи із середовищем розроблення, створити та запустити перший проєкт, а також ознайомитися з його базовою структурою.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Flutter – це відкритий фреймворк від Google для створення кросплатформних застосунків під Android, iOS, Web та Desktop із єдиної кодової бази.

- Використовує власний високопродуктивний рушій рендерингу (Skia);
- Будує інтерфейс на основі віджетів і застосовує декларативний підхід побудови UI;
- Підтримує "гаряче перезавантаження" (*hot reload*) для швидкого внесення змін у код;
- Має багату екосистему плагінів та бібліотек, у тому числі інтеграцію з Firebase.

Dart – це мова програмування, розроблена Google, яка використовується у Flutter як основна.

- Підтримує об'єктно-орієнтоване програмування з класами та інтерфейсами;
- Має потужні можливості асинхронного програмування (*async/await*, *Futures*, *Streams*);
- Компілюється як у машинний код (JIT і AOT), так і у JavaScript (для веб-застосунків).

Середовище розробки. Для розробки кросплатформних застосунків на Flutter найчастіше використовуються Android Studio або Visual Studio Code.

Android Studio забезпечує потужні засоби відлагодження та інтегровані інструменти для Android.

Visual Studio Code – легший редактор із широким вибором розширень для Flutter і Dart. У середовище розробки інтегруються: Flutter SDK, емулятори Android або iOS, термінал і плагіни для швидкої роботи з проєктами.

Установка SDK. Flutter SDK – набір інструментів для створення, компіляції та запуску застосунків.

1. Завантажується з офіційного сайту flutter.dev.
2. Після розпакування потрібно додати шлях до змінної середовища PATH.
3. Для Android потрібна наявність Android SDK, а для iOS – Xcode.
4. Перевірка встановлення виконується командою “flutter doctor”, яка показує наявність і стан усіх потрібних компонентів.

Створення проєкту виконується командою:

```
flutter create my_app
```

У результаті генерується структура з базовим кодом. Для роботи потрібно відкрити проєкт у IDE та налаштувати симулятор або підключити реальний пристрій.

Запуск проєкту здійснюється з терміналу:

```
flutter run
```

або

через кнопку Run у IDE.

Flutter компілює код у нативний байткод для вибраної платформи. Можна запускати як на емуляторах (Android Emulator, iOS Simulator), так і на фізичних пристроях.

Структура проекту. Типовий Flutter-проект має таку структуру:

my_app/

```
android/      # файли для Android-платформи
ios/          # файли для iOS
lib/          # основний код програми (main.dart)
test/         # тести
web/          # веб-версія застосунку
windows/, macos/, linux/  # десктопні платформи
pubspec.yaml  # конфігурація: залежності, ресурси
```

- lib/ – головна директорія з кодом. Точка входу – main.dart;
- pubspec.yaml – описує залежності та ресурси;
- android/, ios/ – містять налаштування для нативних платформ;
- test/ – використовується для unit-тестів.

Файл **pubspec.yaml** – це головний конфігураційний файл проекту на Flutter/Dart, який описує метадані проекту та його залежності. Він обов’язково створюється при ініціалізації нового застосунку.

Основне призначення **pubspec.yaml**:

- Зберігає інформацію про застосунок (назва, версія, опис);
- Визначає зовнішні пакети та плагіни, які потрібні для роботи;
- Містить відомості про ресурси (шрифти, зображення, локалізація);
- Визначає середовище виконання (версії Dart/Flutter).

Ідентифікаційні дані в pubspec.yaml:

name: my_app

description: Перше Flutter-застосування

version: 1.0.0+1

- name — назва пакета/застосунку;
- description — короткий опис;
- version — версія застосунку (<основна>.<додаткова>.<патч>+<build>).

Налаштування середовища в pubspec.yaml:

environment:

```
  sdk: ">=3.0.0 <4.0.0"
```

- Вказує діапазон підтримуваних версій Dart.

Залежності в pubspec.yaml:

dependencies:

```
  flutter:
```

```
    sdk: flutter
```

```
  cupertino_icons: ^1.0.6
```

- dependencies — основні пакети, необхідні для роботи програми;
- dev_dependencies — бібліотеки, що потрібні лише для розробки та тестування (наприклад, flutter_test).

Flutter-специфічні налаштування в pubspec.yaml:

```
flutter:
```

```
  uses-material-design: true
```

```
  assets:
```

```
    - assets/images/
```

```
  fonts:
```

```
    - family: Roboto
```

```
      fonts:
```

```
        - asset: assets/fonts/Roboto-Regular.ttf
```

- uses-material-design — підключення іконок Material Design;
- assets — ресурси (зображення, звуки тощо);
- fonts — власні шрифти.

Взаємозв'язок pubspec.yaml з іншими файлами:

- pubspec.yaml визначає які пакети потрібні;
- pubspec.lock фіксує, які конкретні версії цих пакетів встановлені;
- Команда “flutter pub get” читає pubspec.yaml і завантажує пакети у кеш.

Основи мови Dart і побудова віджетів у Flutter. Код застосунку розміщується у файлах із розширенням .dart у каталозі lib. Точкою входу в програму є функція main():

```
void main() {  
  runApp(const MyApp());  
}
```

Функція runApp() приймає кореневий віджет і відображає його на екрані. У Flutter весь інтерфейс складається з віджетів: текст, зображення, кнопки, поля введення, панелі та елементи компоновання також є віджетами.

Класи та створення об'єктів у Dart. Клас описує структуру і поведінку об'єкта. Конструктор класу використовується для створення його екземплярів:

```
class AppInfo {  
  final String title;  
  final String description;  
  
  const AppInfo({  
    required this.title,  
    required this.description,  
  });  
}
```

Ключове слово final означає, що значення змінної задається один раз. Ключове слово const дає змогу створювати незмінні об'єкти під час компіляції. Іменованний параметр із позначенням required є обов'язковим.

StatelessWidget і StatefulWidget. Для створення власних елементів інтерфейсу використовують два основні типи віджетів: StatelessWidget – це віджет, стан якого не змінюється під час роботи; StatefulWidget – це віджет, стан

якого може змінюватися внаслідок дій користувача або інших подій. Для створення простого статичного стартового екрана достатньо використати StatelessWidget:

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      home: HomeScreen(),  
    );  
  }  
}
```

Метод build() формує та повертає дерево віджетів, яке Flutter відображає на екрані. Параметр BuildContext містить інформацію про розташування віджета в цьому дереві.

Для побудови простого екрана можна використати такі віджети:

- MaterialApp – це кореневий віджет застосунку з підтримкою Material Design;
- Scaffold задає базову структуру екрана;
- AppBar – це верхня панель із заголовком;
- Center задає розташування дочірнього віджета по центру;
- Column задає вертикальне розташування кількох віджетів;
- Text – це віджет відображення тексту;
- Image.asset – це віджет відображення локального зображення;
- SizedBox віджет для створення відступу між елементами.

Приклад простого тематичного екрана:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: HomeScreen(),  
    );  
  }  
}
```

```
class HomeScreen extends StatelessWidget {  
  const HomeScreen({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Домашня комора'),  
      ),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,
```

```

children: [
  Image.asset(
    'assets/images/app_image.png',
    width: 160,
    height: 160,
  ),
  const SizedBox(height: 20),
  const Text(
    'Облік домашніх запасів продуктів',
    textAlign: TextAlign.center,
  ),
],
),
),
);
}
}

```

Властивість `children` містить список дочірніх віджетів. Елементи списку відокремлюються комами. Якщо віджет та значення його параметрів не змінюються, перед його конструктором доцільно використовувати `const`.

Підключення локального зображення. Файл зображення потрібно помістити, наприклад, у каталог: `assets/images/app_image.png`. Після цього каталог необхідно зареєструвати у файлі `pubspec.yaml`:

```

flutter:
  uses-material-design: true

assets:
  - assets/images/

```

У файлі pubspec.yaml важливо дотримуватися правильних відступів. Після внесення змін потрібно виконати команду: flutter pub get.

Зображення додається до інтерфейсу за допомогою конструктора:

```
Image.asset('assets/images/app_image.png')
```

Якщо шлях у коді не відповідає фактичному розташуванню файлу або ресурс неправильно вказано в pubspec.yaml, Flutter не зможе відобразити зображення.

Необхідні програмні засоби для виконання завдання

Для виконання цієї роботи потрібно встановити:

1. Середовище розробки Android Studio або текстовий редактор Visual Studio Code з необхідними плагінами для підтримки Flutter та Dart;
2. Flutter SDK;
3. Android SDK (не обов'язково, якщо не планується підтримка Android);
4. Xcode (не обов'язково, якщо не планується підтримка iOS);

Для ознайомлення із мовою Dart і її засобами рекомендується ознайомитись із онлайн-редактором коду <https://dartpad.dev>.

Варіанти завдань

Варіант завдання відповідає порядковому номеру студента у списку групи.

1. **Домашня комора.** Облік продуктів, їх кількості, категорій і термінів придатності.
2. **Менеджер гарантій і покупок.** Збереження інформації про придбані товари, гарантійні строки та чеки.
3. **Журнал обслуговування автомобіля.** Облік ремонтів, пробігу, заміненних деталей і витрат на автомобіль.

4. **Помічник із догляду за рослинами.** Каталог рослин із фотографіями, правилами догляду та журналом виконаних процедур.
5. **Органайзер домашніх справ.** Створення і розподіл побутових завдань зі строками та статусами виконання.
6. **Каталог навчальних матеріалів.** Збереження конспектів, посилань і файлів за дисциплінами та темами.
7. **Трекер пошуку роботи.** Облік вакансій, компаній, співбесід та етапів розгляду заявок.
8. **Планувальник фотосесій.** Планування зйомок із зазначенням дати, локації, учасників і необхідних матеріалів.
9. **Менеджер позичених речей.** Облік речей, позичених або переданих іншим людям, із датами повернення.
10. **Органайзер волонтерських заходів.** Створення заходів, ведення списку учасників і розподіл завдань.
11. **Щоденник велосипедиста.** Облік поїздок, технічного стану велосипеда та виконаного обслуговування.
12. **Планувальник ремонту квартири.** Організація ремонтних робіт за приміщеннями, етапами, строками та витратами.
13. **Каталог настільних ігор.** Облік настільних ігор, зіграних партій, учасників та результатів.
14. **Органайзер невеликих подій.** Планування подій зі списком гостей, завданнями та основною інформацією про проведення.
15. **Трекер поштових відправлень.** Ведення списку посилок, номерів відправлень і поточних статусів доставки.
16. **Моніторинг комунальних витрат.** Збереження показників лічильників, обсягів споживання та вартості комунальних послуг.
17. **Планувальник садових робіт.** Облік рослин, ділянок і запланованих сезонних робіт.
18. **Менеджер особистих документів.** Каталог документів із категоріями, строками дії та електронними копіями.

19. **Щоденник риболовлі.** Записи про виїзди, водойми, погодні умови, снасті та улов.
20. **Органайзер навчального гуртка.** Ведення розкладу зустрічей, списку учасників, завдань і навчальних матеріалів.
21. **Трекер чергувань.** Створення графіка чергувань, призначення виконавців і фіксація виконання.
22. **Каталог домашніх інструментів.** Облік інструментів, місць їх зберігання, стану та історії використання.
23. **Планувальник гардероба.** Каталог одягу з категоріями, фотографіями та можливістю формувати комплекти.
24. **Каталог закладів і відгуків.** Збереження відвіданих закладів, особистих оцінок, нотаток і фотографій.
25. **Менеджер спільних витрат.** Облік витрат групи користувачів і визначення частки кожного учасника.
26. **Органайзер творчих проєктів.** Планування творчих робіт із поділом на етапи, матеріали та строки виконання.
27. **Трекер навчальної практики.** Щоденні записи про виконані завдання, витрачений час і набуті навички.
28. **Система бронювання обладнання.** Каталог спільного обладнання та облік його бронювання користувачами.
29. **Електронне бюро знахідок.** Публікація оголошень про загублені й знайдені речі з описом і фотографіями.
30. **Тренажер мовних карток.** Створення тематичних наборів слів і фраз та відстеження результатів повторення.

Завдання

1. Визначте варіант завдання, що відповідає вашому порядковому номеру у списку групи. Вкажіть номер варіанта і назву застосунку у звіті.
2. Встановіть середовище розроблення та Flutter SDK. У звіті відобразіть результат виконання команди: **flutter doctor**.

3. Створіть Flutter-застосунок із назвою, що відповідає вашому варіанту завдання. У звіті відобразіть результат виконання команди: **flutter create**.
4. Ознайомтеся зі структурою створеного проєкту. У звіті коротко опишіть призначення каталогів **lib**, **android**, **ios**, **web**, **test**, а також файлів **main.dart** і **pubspec.yaml**.
5. У файлі **pubspec.yaml** змініть опис застосунку в полі **description** та підключіть одне локальне тематичне зображення.
6. У базовому демонстраційному проєкті замініть стандартну назву, заголовки й текстові написи відповідно до тематики застосунку. Додайте на головний екран підключене тематичне зображення.
7. Виконайте команду: **flutter run**. У звіті покажіть вигляд застосунку в Android-емуляторі, iOS-симуляторі, на фізичному пристрої або у вікні браузера залежно від вибраної цільової платформи.

Вимоги до звіту

Захист лабораторних робіт включає також виконання звіту у текстовому редакторі. Рекомендації до форматування звіту: розмір шрифту основного тексту – 14 пт; міжрядковий інтервал – 1,25; абзацний відступ зліва – 1 см; перед- й післяабзацні міжрядкові інтервали – 0; вирівнювання основного тексту – двостороннє; заголовки відцентровані з накресленням – Bold, усі поля сторінки – 2 см, орієнтація аркушів книжкова. Аркуші звіту формату А4 нумеруються без вказання номера на першій сторінці.

Складовими звіту є титульний аркуш, тема, мета, постановка завдання, отримані результати, висновки.

У розділі «Отримані результати» розмістити відповіді на питання завдання.

У висновках зазначити й охарактеризувати отримані результати в ході виконання лабораторної роботи.

Контрольні питання для самоперевірки

1. Що таке Flutter і для яких платформ він дозволяє створювати застосунки?
2. Який рушій використовується у Flutter для рендерингу інтерфейсу?
3. Які особливості Flutter фреймворку вам відомі?
4. Яка перевага функції "гаряче перезавантаження" (*hot reload*)?
5. Яка мова програмування використовується у Flutter?
6. Які основні особливості мови Dart вам відомі?
7. У яких формах може компілюватися Dart-код?
8. Які середовища розробки найчастіше застосовують для Flutter? У чому їхні особливості?
9. Яка команда дозволяє перевірити правильність встановлення SDK і всіх компонентів?
10. Якою командою створюється новий Flutter-проект?
11. Як можна запустити Flutter-застосунок?
12. Назвіть основні директорії та файли у структурі Flutter-проекту.
13. Яке призначення файлу pubspec.yaml і яку інформацію він зберігає?
14. У чому різниця між dependencies та dev_dependencies у pubspec.yaml?
15. Як підключити та відобразити локальне зображення у Flutter-застосунку?
16. Який взаємозв'язок між файлами pubspec.yaml та pubspec.lock?
17. Яка команда використовується для завантаження залежностей, визначених у pubspec.yaml?
18. Яка функція є точкою входу у Flutter-застосунок і для чого використовується функція runApp()?
19. Що таке віджет у Flutter? Наведіть приклади базових віджетів.
20. Яке призначення методу build() у класі віджета?

Список рекомендованої літератури

1. Richard Rose: Flutter and Dart Cookbook: Developing Full-Stack Applications for the Cloud 1st Edition / O'Reilly Media – 2023. – 308 p.
2. Rap Payne: Flutter App Development: How to Write for iOS and Android at Once – 2024. – 340 p.

3. Flutter [Электронный ресурс]. - Режим доступа: <https://flutter.dev/>